



C avancé

TP2 - Gestion avancée de la mémoire

Maxime MARIA

Respectez bien les consignes et testez le bon fonctionnement de tous vos programmes ! Pensez aussi à sauvegarder régulièrement ainsi qu'à garder tous vos codes !

1 Espace mémoire d'un programme

1. Réalisez les mêmes tests que dans la partie « les segments mémoire : la zone statique » du cours (en utilisant la commande `size` dans votre terminal). Pour cela, vous devez compiler avec l'option `-c` et lancer la commande sur le fichier `.o` qui a été produit. Analysez et comprenez les résultats.
2. Déclarez plusieurs variables dans la pile et affichez leur adresse. Analysez et comprenez les résultats.
3. Déclarez plusieurs variables sur le tas et affichez leur adresse. Analysez et comprenez les résultats.

2 Little or big endian ?

1. Écrivez une fonction `printMemoryRepresentation` qui affiche un à un les octets d'une variable passée en paramètre (utilisez une notation hexadécimale sur deux caractères : `%.2x`).
2. Appelez votre fonction avec la valeur entière `0x01234567`. Dans quel ordre sont rangés vos octets ?
3. Modifiez la fonction pour qu'elle affiche les bits associés à chaque octet (vous pouvez utiliser la fonction que vous avez codé au TP 2, exercice 2, question 3 de BLC).
4. Rappelez votre fonction avec la valeur entière `0x01234567`. Dans quel ordre sont rangés les bits vos octets ?

3 Tableau dynamique d'entier

Dans cet exercice, vous allez implémenter un tableau dynamique (*i.e.* dont la taille n'est pas fixe) contenant des entiers. Téléchargez le fichier `vector.h` depuis la plateforme pédagogique. Celui-ci contient une structure `IntVector` :

```
1 typedef struct {
2     int *data; // Tableau contenant les entiers
3     size_t size; // Nombre d'elements dans le tableau
4     size_t capacity; // Nombre d'elements pre-alloue
5 } IntVector;
```

La capacité (`capacity`), correspond à une zone mémoire pré-allouée. Lorsqu'on initialise un `IntVector` (`vectorInit`) avec une taille de X , on alloue dynamiquement un tableau de taille $X \times 2$. Par contre, seul les `size` premiers éléments sont accessibles à l'utilisateur. Ensuite, lorsque le tableau atteint une taille (`size`) égale à sa capacité, il faut ré-allouer `data` de sorte à ce que sa capacité soit égale à deux fois sa taille.

Ici, c'est à l'utilisateur de s'assurer de la validité des indices données aux fonctions, nous ne gérons pas ces erreurs. Par contre, il faut gérer les erreurs liées à la mémoire (vos fonctions retournent 0 quand tout s'est bien passé et -1 sinon).

L'objectif de cet exercice est de vous faire utiliser les fonctions de gestion de la mémoire ! La seule boucle que vous devrez écrire est pour la fonction `vectorPrint`.

1. Implémentez les fonctions dont les prototypes sont donnés dans `vector.h`.

4 Manipulation de caractères

Écrivez une fonction `void intToString(int number, char str[])` qui convertit l'entier `number` en une chaîne de caractères `str`. Bien entendu, vous devez programmer cette fonction et pas seulement appeler `itoa`. Pensez au caractère de fin de chaîne de caractères !

N.B. : votre programme doit aussi fonctionner avec des nombres négatifs.

5 Chaînes de caractères et `string.h`

1. Écrivez un programme qui prend en paramètres deux chaînes de caractères, un prénom et un nom, et les affiche dans la console, séparés d'un espace.
2. Déclarez et initialisez une chaîne de caractères de sorte à ce qu'elle contienne une adresse mail telle que `prenom.nom@unilim.fr` (en utilisant le prénom et le nom précédents). Vous n'allouerez que la mémoire strictement nécessaire. Vous n'utiliserez que les fonctions de `string.h`.
3. Utilisez la fonction `strcspn` pour trouver la longueur du prénom à partir de la chaîne précédente.
4. Trouvez la longueur du nom à partir de la chaîne précédente.